

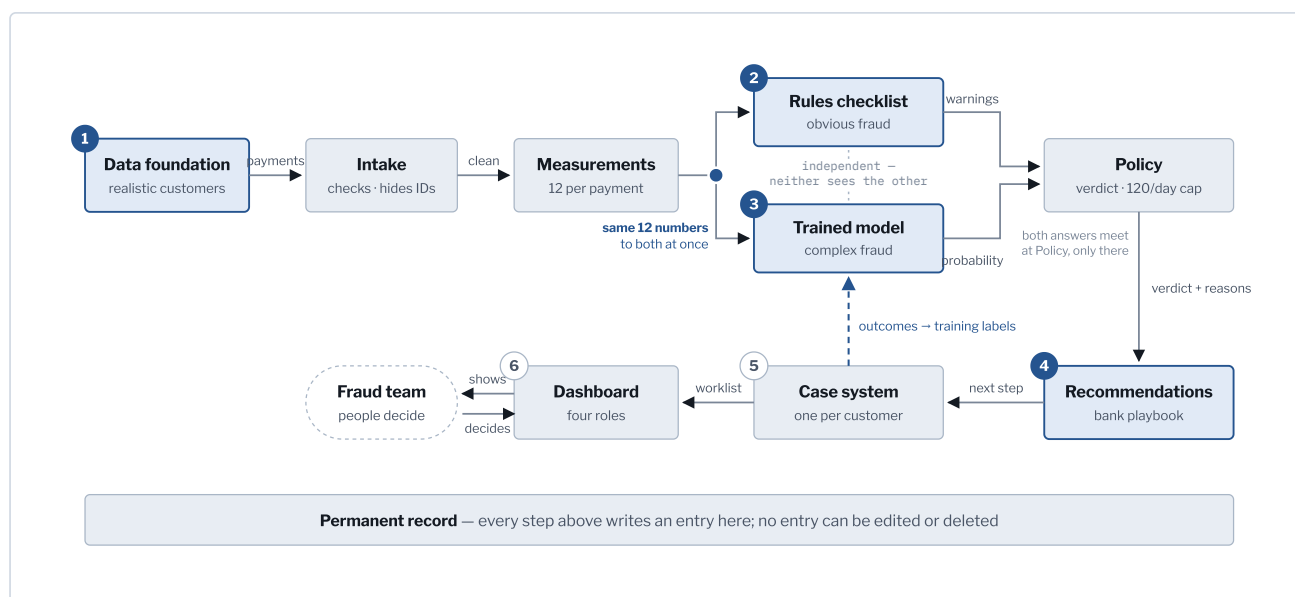
Risk Radar Blueprint

What we are building, in six parts. What each part is for, what it must do, and the number that tells us whether it is doing it.

Scope statement. Risk Radar watches a Nigerian bank's stream of payments, spots the ones that look like fraud, and puts them in front of a fraud analyst with a clear recommendation of what to do next. It **advises**; a person decides. It never blocks, holds or reverses a payment.

It is built from six parts: **realistic data** to learn from, a **checklist of written rules** for fraud that is obvious, a **trained model** for fraud that is not, a **recommendation engine** that turns the evidence into a next step, a **case system** with a permanent record, and a **dashboard** the fraud department works in all day.

How the parts connect



Every payment runs left to right along the top, and the result runs back along the bottom to the people who act on it. At the fork (the blue dot), the *same* twelve measurements go to the rules and to the model at once. Neither waits for the other or sees the other's answer, which is what lets us measure what each one catches on its own. Their two answers meet only at the policy step, which decides; they are never blended into one number. The dashed blue line is the loop that matters most in the long run. When an analyst records "real fraud" or "false alarm", that answer becomes a training label, which is how the model eventually stops learning only from invented data.

1

PART ONE

Data foundation — a realistic financial dataset, built with context

Objective: give the rules and the model believable Nigerian payment behaviour to learn from, with fraud that hides among ordinary life rather than standing apart from it.

No bank's real transactions are available, and no public Nigerian fraud dataset exists. So we generate customers — ordinary people, salaried workers, shop owners, traders — who each own accounts, keep habits and pay people, then have some of them robbed in three different ways. Real published statistics shape the result: our fraud rate is tuned to the one Nigeria's interbank settlement body reports.

Must do

- Model customers owning several accounts, so one person's incident stays one investigation.
- Give legitimate customers the behaviours fraud also has — paying new people, paying many people in an hour, occasional large sums — so

fraud has something to hide in.

- Fail its own check if any single measurement can separate fraud from normal by itself.

MEASURE	TARGET	NOW	STATUS
Fraud rate	≈ 0.30% (NIBSS)	0.296%	MET
Fraud incidents to test on	≥ 600	604	MET
Measurements that give fraud away on their own	0 over 15%	0 · one at 5.6%, explained	MET
Checked against published channel and state figures	all key shares	fraud rate only	PARTLY

2

PART TWO

Rules checklist — fraud that is obvious

Objective: catch the fraud a person would recognise instantly, with near-certain accuracy, and catch it even when it is a new variant the model has never seen.

Plain written rules, each producing a named warning with its evidence — never a score. Some raise the risk (a burst of payments, a run of refused card attempts). Some override everything (a sanctioned destination). Some lower it (a long-standing payee receiving a normal amount). Settings change from the admin screen, with every change recorded.

Must do

- Explain itself: every warning says what fired and why.
- Stay inside the team's capacity: no rule may spend more of the daily alert budget than it earns.
- Be tested one rule at a time, and drop any rule the data does not support.

MEASURE	TARGET	NOW	STATUS
Card-testing rule: how often it is right	≥ 0.95	0.998	MET

MEASURE	TARGET	NOW	STATUS
Card testing caught, new variant, rules alone	≥ 0.90	192 of 194 · 0.99	MET
Payment-burst rule: share of daily budget	$\leq 60\%$	~28% (5+ in an hour, to a destination new to the bank)	MET
New mule fan-out caught by the rules alone	as high as budget allows	0.87 (was 0.41)	MET
"Familiar payee" rule firing where there is no payee	never	never (was 84% of its firings)	FIXED
Every rule measured on its own	all six	three	PARTLY

3

PART THREE

Trained model — fraud that is complex, calibrated to tell the truth

Objective: catch fraud that no single rule describes — the combination of a new device, a quiet pause and a rush of transfers — and give a probability that means exactly what it says.

Hundreds of small decision trees, each correcting the last, trained on older data and then adjusted on newer data it never saw, so that "0.30" really means about three in ten. It is one voice, not the decision: on its own it catches only what resembles its training, which is why the rules sit beside it.

Must do

- Be tested on a kind of fraud hidden from its training, not only on more of the same.

- Produce an honest probability, because the daily workload is predicted from it.
- Use the same twelve measurements live as in training, checked by a test.

MEASURE	TARGET	NOW	STATUS
Account takeover caught, never seen in training	≥ 0.70	0.78 (rules alone: 0.20)	MET
Honesty of its probability (stated ÷ actual)	≥ 0.90	0.92	MET
Calibration error where alerts are raised	≤ 0.04	0.032	MET
Same numbers in training and live	identical	identical, tested	MET
Useful on its own	—	0.40 · below one raw measurement	BY DESIGN

4

PART FOUR

Recommendation engine — the next step, in the bank's own procedure

Objective: turn the evidence into one clear next action and its reason — "block the card and reissue: 14 refused small attempts" — so an analyst never has to work out what to do from raw data.

It reads the verdict, the warnings, the money at risk and the customer's pattern, and picks the step from a playbook. It is deterministic and written down, never learned, so every recommendation can be explained in the words that produced it.

On "real bank policy". No bank has shared its fraud procedures with us, so the playbook today is our own, written from how Nigerian fraud desks describe their work. It is built as a lookup the bank can replace: their procedure in, their steps out, with no code change. We will not describe it as a bank's policy until one is loaded.

Must do

- Give one action, numbered next steps, and the reason, for every case.
- Say how urgent it is, tied to a response clock.
- Record whether the analyst followed it, so the advice can be checked against real outcomes.

MEASURE	TARGET	NOW	STATUS
Cases with an action, steps and reason	100%	100%	MET
Bank's own procedure loaded	yes	our playbook, bank-replaceable	PARTLY
How often analysts follow the advice	tracked weekly	recorded, not yet reported	NOT YET

5

PART FIVE

Case system and permanent record — what holds it together

Objective: turn many alerts about one customer into one case, keep the team inside its capacity, and prove afterwards who did what.

- Group alerts about the same customer into one case, so an attacker moving between someone's accounts stays one investigation.
- Hold the whole system to the daily alert budget — rules first, the model gets what is left.
- Record every decision and action permanently; administrators may tune the system but can never open a case.

MEASURE	TARGET	NOW	STATUS
Time from payment to decision (95% of the time)	under 2 s	0.88 s	MET
Payments lost in a 5× burst	0	0	MET
Alerts per day, whole system	≤ 120	82 in the latest demo day	MET
Record edits possible	none	none · tamper-evident	MET

What the over-budget day turned out to be. An earlier demo day ran 28% over the budget. The cause was the demo builder, not the system: its live day was generated as a different bank with no history, and was counted on top of a day already loaded. With that fixed, the rebuilt demo day raised 82 alerts and caught both real incidents in it. One caveat stays open: the demo bank remembers only about three days, so the payment-burst rule fires more there than it would in a real bank with years of history.

6

PART SIX

Dashboard — where the department works

Objective: let each person in the fraud department do their day's job from one screen: analysts clear cases, the lead sees whether the desk is coping, security handles break-ins, the administrator tunes settings.

Fraud analyst

- The next case handed to them, largest risk first
- Money at risk, the recommendation, why it fired
- One key to record the outcome

Fraud operations lead

- Backlog, waiting times, who carries what
- How many alerts turned out to be false
- Signs off analysts' outcomes

Information security

- Cases showing a break-in — new device, dormant account
- Escalations routed to them

Administrator

- Rules, thresholds, models, the record
- No access to any case, by design

MEASURE	TARGET	NOW	STATUS
Keystrokes to record an outcome	1	1	MET
Cases past their response clock	under 10%	not yet measured with real users	NOT YET
Median time to decide a case	under 3 min	not yet measured	NOT YET

The number that sums it up

Shown a kind of fraud hidden from its training, with the team held to 120 alerts a day, the whole system catches **about nine incidents in ten** (0.89): account takeover 0.78, mule fan-out 0.91, card testing 0.99. The model is what catches account takeover; the rules catch card testing and most mule fan-out, now that the payment-burst rule looks at where the money is going, not only how fast.

We can claim the method is sound and measured. We cannot yet claim it works on real criminals, because we have never seen any. Running it alongside a real bank for a month, alerting nobody, is what would settle that.

Out of scope

- Blocking, holding or reversing payments — advisory only, permanently.
- Connecting to a real bank's systems — a separate project.
- Detecting networks of linked criminals — a later version.
- Testing a new model on live traffic before switching to it — designed, not built.

Who owns which part

PART	OWNER
1 · Data foundation, and testing each rule on the data	Ire
2, 3 · Rules and model code · 5 · intake, measurements, policy, case system, record	Chidera
4 · Recommendation playbook	Chidera, with the team
6 · Dashboard	Paul
Plan, tests, documents, presentation	Kanyinsola

Figures from the project's own measurements on 11 September 2026: `system-evaluation.json`, `calibration.json`, `budget-grid.json`, `own-corpus-audit.json` and the load test. Each decision behind them is in the project decision log. Companions: *What We Are Building* and *What We Learned From Testing Both Approaches*.